Clustering Algorithms Report

In this report, many clustering algorithms are implemented such as K-Means, OPTICS, K-Modes and Divisive Clustering.

These algorithms will be explained theoretically and with python examples.

1.OPTICS Clustering

OPTICS (Ordering Points To Identify the Clustering Structure) is a density-based clustering algorithm similar to DBSCAN clustering. Unlike DBSCAN which struggles with varying densities. OPTICS does not directly assign clusters but instead creates a reachability plot which visually represents clusters.

Core Distance: Minimum distance needed for a point to be defined as core point. If a point does not have enough neighbours, its core distance is undefined.

Reachability Distance: It is a measure of how difficult it is to reach from one point to another. It is calculated as the larger core distance of the starting point and the actual point.

MinPts = 5 means that point must have at least 5 nearby neighbours to be defined as core point.

The core distance of point p is 3mm meaning it needs at least 5 points within a 3mm radius to be considered as a core point.

The reachability distance from q to p is 7mm (since q is farther than p's core distance).

The reachability distance from r to p is 3mm (since r is within p's core distance).

Steps of OPTIC Clustering

1. The algorithm selects a starting point and checks it has at least MinPts neighbors within Eps.

2. If the point matches the density requirements, it is marked as a core point and its nearby points are analyzed.

3. Reachability distance is computed for each point.

4. Points are then processed in order of their reachability distance hence forming reachability plot.

5. Clusters appears as valleys i.e low reachability distances and noise appears as peaks i.e high reachability distances.

Here is a python program that explains how OPTICS Clustering is implemented:

optic_python.py × OPTICS Clustering Using Synthetic Data.png

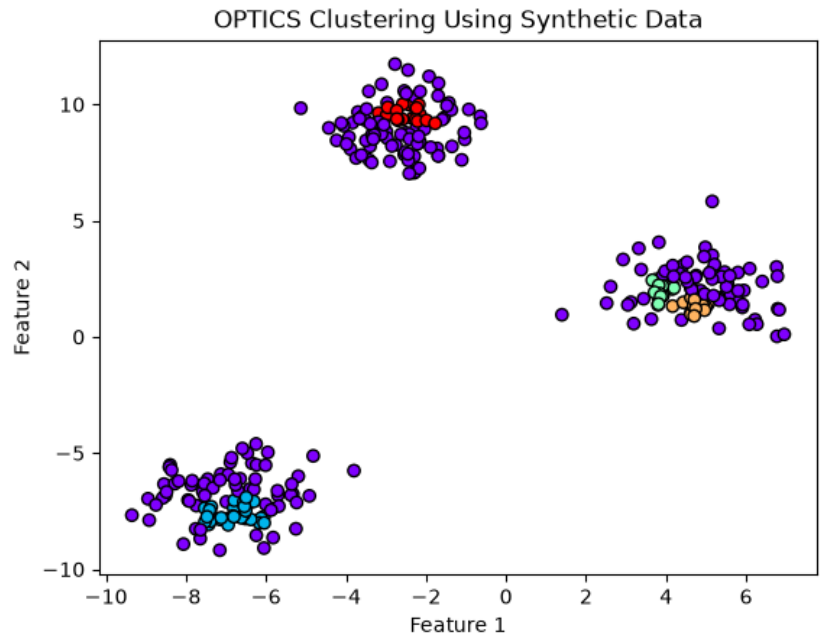
```
1
2   from sklearn.cluster import OPTICS
3   from sklearn.datasets import make_blobs
4   import matplotlib.pyplot as plt
5
6   X, Y = make_blobs(n_samples=300, centers=3, random_state=42)
7
8   clustering = OPTICS(min_samples=5, xi=0.05, min_cluster_size=0.05)
9   clustering.fit(X)
10
11  labels = clustering.labels_
12
13  plt.scatter(X[:,0], X[:,1], c=labels, cmap='rainbow', edgecolor='k')
14  plt.title("OPTICS Clustering Using Synthetic Data")
15  plt.xlabel("Feature 1")
16  plt.ylabel("Feature 2")
17  plt.savefig("OPTICS Clustering Using Synthetic Data.png")
18  plt.show()
```

Here, scikit-learn library used to create synthetic dataset and OPTICS library is added to the program.

İn OPTICS() Function we defined min_samples as 5 which means MinPts, amount needed to form core point.

The xi=0.05 parameter controls how significant a change in density must be for OPTICS to detect a cluster boundary.

Output of the graph is at the next page:



Plot is scattered and each cluster label had different color.

This example demonstrates how OPTICS can be used to identify clusters in an unlabeled dataset.

1.2 OPTICS Program with Real Dataset

In this example **Brazilian E-Commerce Public Dataset by Olist** is used to implement OPTICS Clustering. Dataset is downloaded from kaggle.com.

olist_geolocation_dataset.csv is used here to find locations of the Brazilian zip codes to lat/lng coordinates.

Pandas library used for reading the dataset. OPTICS from scikit-learn is implemented for creating OPTIC Clustering algorithm.

For creating real map, geopandas and contextility is used.

The numpy library is used for mathematical operations, including converting coordinates from degrees to radians.

Box from the Shapely library is used to define the geographical boundaries of Brazil.

In this program, there are many outputs which will be explained later on.

Full Python program at the next page:

Geographical Clustering of Brazilian Postal-Code Areas Using OPTICS

python_code.py | brazil1.png | brazil.png | reachability_plot.png | olist_geolocation_dataset.csv

```python
import pandas as pd
from sklearn.cluster import OPTICS
import matplotlib.pyplot as plt
import geopandas as gpd
import contextily as cx
import numpy as np
from shapely.geometry import box

df = pd.read_csv('olist_geolocation_dataset.csv')

pd.set_option('display.max_columns', 5)
pd.set_option('display.max_rows', 50)
pd.set_option('display.max_colwidth', 300)
pd.set_option('display.width', 250)

print("\nNumber of rows in the dataset: ", df.shape[0])
print("Number of columns in the dataset: ", df.shape[1])

print("\nFirst 5 rows of the dataset: ")
print(df.head())

print("\nLast 5 rows of the dataset: ")
print(df.tail())

print("\nMissing values in the dataset: ")
print(df.isnull().sum())

print("\nDataset Information: ")
print(df.info())

print("\nDataset Description: ")
print(df.describe())

df = df.drop_duplicates()
df = df.dropna(subset=['geolocation_lat', 'geolocation_lng'])
df = df[df['geolocation_lat'].between(-90,90) & df['geolocation_lng'].between(-180,180)]
```

Libraries are added, data is read and dataset information is printed out.

Missing values are checked and found out there arent any.

Duplicate rows are dropped out and rows where latitude and longitude columns are empty are dropped from the dataset.

Latitude is between {-90,90} and Longitude is between {-180,180}.

```
37
38  print("\nNumber of unique postal-code prefixes: ")
39  print(df['geolocation_zip_code_prefix'].nunique())
40
41  print("\nNumber of repeated postal-code prefixes: ")
42  print(df.duplicated(subset=['geolocation_zip_code_prefix']).sum())
43  # Large difference between number of unique postal-code prefixes and Number of repeated postal-code prefixes
44  # means that many prefixes has multiple geographical records
45
46  # são paulo becomes sao paulo / make sure city names are same
47  df['geolocation_city'] = df['geolocation_city'].str.strip()
48  df['geolocation_city'] = df['geolocation_city'].str.lower().str.normalize("NFKD")
49  df['geolocation_city'] = df['geolocation_city'].str.encode('ascii', errors='ignore').str.decode("utf-8")
50
51  df['geolocation_state'] = (df['geolocation_state'].str.upper().str.normalize("NFKD").
52                             str.encode('ascii', errors='ignore').str.decode("utf-8"))
53
54  print("\nRows where city name is just initials")
55  print(df[df['geolocation_city'] == "sp"])
56  print("\n")
57
58  df.loc[(df['geolocation_city'] == "sp") & (df["geolocation_state"] == "SP"), "geolocation_city"] = "sao paulo"
59  print(df[df['geolocation_city'] == "sp"]) # outputs empty dataframe
60
61  print("\nFirst 25 rows of the Dataset after converting city and state names")
62  print(df.head(25))
63
64  cities_per_zip_prefix = df.groupby("geolocation_zip_code_prefix")['geolocation_city'].nunique()
65  states_per_zip_prefix = df.groupby("geolocation_zip_code_prefix")['geolocation_state'].nunique()
66
67  zip_with_multiple_cities = cities_per_zip_prefix[cities_per_zip_prefix > 1]
68  zip_with_multiple_states = states_per_zip_prefix[states_per_zip_prefix > 1]
69
70  print("\nPostal code prefixes that have multiple cities: ")
71  print(zip_with_multiple_cities)
72  print("\nPostal code prefixes that have multiple states: ")
73  print(zip_with_multiple_states)
```

Number of unique postal code prefixes are printed out.

Number of repeated postal code prefixes are printed out.

A large number of duplicated prefix records indicates that many postal-code prefixes are represented by multiple geographical observations.

City names are standardized. City names may contain inconsistent capitalization, additional spaces, or accented characters. These differences can cause the same city to be treated as multiple categories.

For example: São Paulo, são paulo, SAO PAULO, sao paulo, these values refer to the same city but are not identical as strings.

If the city names are just initials (sp), it is converted to sao paulo.

After that, we find the amount of cities and states each prefix have.

Then, we found prefixes that has more than one cities and states.

```
print("\nNumber of Postal code prefixes that have multiple cities ")
print(len(zip_with_multiple_cities))
print("\nNumber of postal code prefixes that have multiple states: ")
print(len(zip_with_multiple_states))

most_frequent_city = df['geolocation_city'].value_counts().idxmax()
print("\nMost frequently shown city in the dataset")
print(most_frequent_city)

most_frequent_state = df['geolocation_state'].value_counts().idxmax()
print("\nMost frequently shown state in the dataset")
print(most_frequent_state)
```

Number of prefixes that have multiple cities and states are printed out.

Most frequently shown city and state is printed out which is sao paulo.

```
df_aggregated = (df.groupby('geolocation_zip_code_prefix').agg(geolocation_lat = ("geolocation_lat", "median"),
                                                               geolocation_lng = ("geolocation_lng", "median"),
                                                               geolocation_city = ("geolocation_city", lambda values :{mode} : values.mode().iloc[0]),
                                                               geolocation_state = ("geolocation_state", lambda values :{mode} : values.mode().iloc[0])
                                                               ).reset_index()
)
```

The dataset is aggregated according to the geolocation_zip_code_prefix column because the same postal-code prefix may appear in multiple rows with different coordinates, city names, or state values.

For the latitude and longitude columns, the median value is calculated.

For the city and state columns, the most frequently occurring value is selected using the mode() function.

So we basically find each zip codes most used cities and its middle ground for its location.

```
print("\nFirst 10 rows of the aggregated dataset: ")
print(df_aggregated.head(10))
print("\nAggregated dataset shape: ")
print(df_aggregated.shape)

# Number of unique postal code prefixes must be same to number of rows
print("\nNumber of unique postal code prefixes: ")
print(df_aggregated['geolocation_zip_code_prefix'].nunique())

print("\nNumber of duplicated postal-code prefixes:")
print(df_aggregated.duplicated(subset=['geolocation_zip_code_prefix']).duplicated().sum())
```

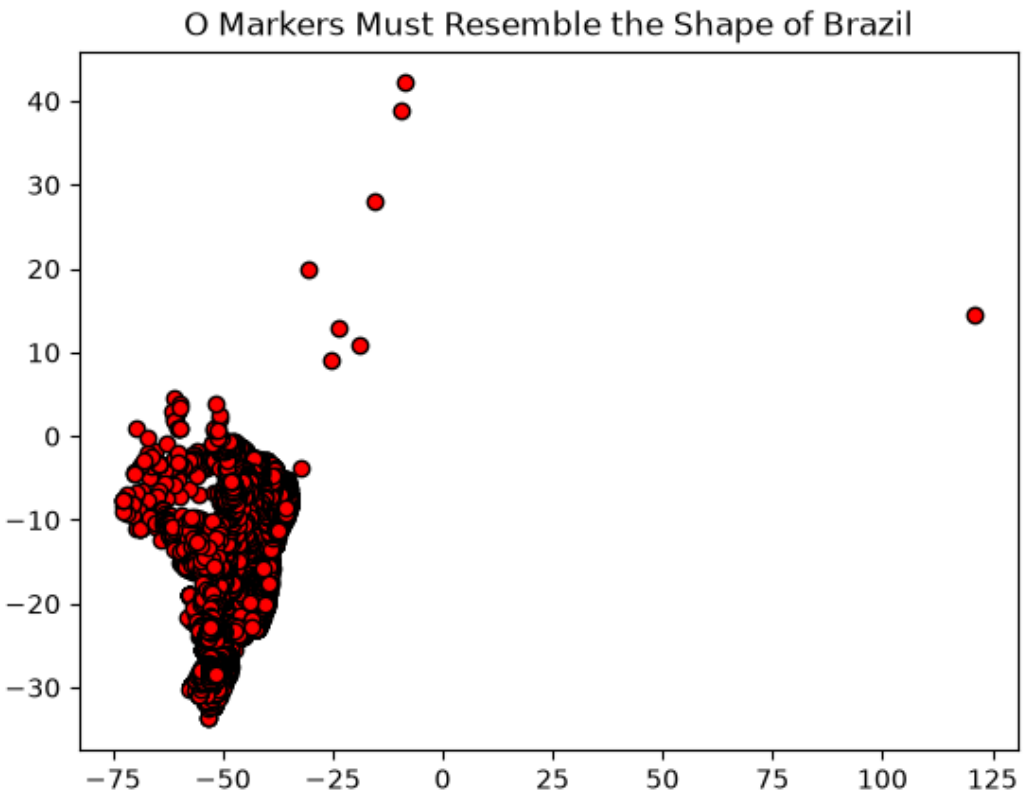
First 10 rows of the aggregated dataset and its shape is printed out and number of unique prefixes are printed out which it must be equal to amount of rows in the aggregated dataset.

Number of duplicaed postal-code prefixes are printed out.

```
print("\nNumber of duplicated coordinate pairs:")
print(df_aggregated.duplicated(subset=['geolocation_lat', 'geolocation_lng']).sum())

print("\nAre there any missing values in the aggregated dataset?")
print(df_aggregated.isnull().sum())

print("\nLatitude range:")
print(df_aggregated["geolocation_lat"].min(), "/", df_aggregated["geolocation_lat"].max())

print("\nLongitude range:")
print(df_aggregated["geolocation_lng"].min(), "/", df_aggregated["geolocation_lng"].max())

print("\nFirst and last 5 rows of the aggregated dataset: ")
print(df_aggregated.head(5).tail(5))

plt.scatter(df_aggregated["geolocation_lng"], df_aggregated["geolocation_lat"], marker='o', c='red',
            edgecolors='black')
plt.title("O Markers Must Resemble the Shape of Brazil")
plt.savefig("brazil1.png")
plt.show()
```

Latitude and longitude range is printed out to make sure it has no outlier values.

Plot is scattered using langitude and longitude. It must be resemble the shape of brazil but its not really detailed plot. Here is how it looks:



```
127
128     coordinates = df_aggregated[["geolocation_lat", "geolocation_lng"]].to_numpy()
129
130     print("\nCoordinate Data:")
131     print(coordinates[:5])
132
133     print("\nCoordinate Shape:")
134     print(coordinates.shape)
135
136     coordinates_radians = np.radians(coordinates)
137
138     print("\nCoordinates in  Radians")
139     print(coordinates_radians[:5])
140
141     min_sample = 20
142     min_cluster_size = 50
143     earth_radius = 6371.01
144
145     optic = OPTICS(
146         min_samples=min_sample,
147         min_cluster_size=min_cluster_size,
148         metric="haversine",
149         xi=0.05,
150         cluster_method="xi"
151     )
152
153     labels = optic.fit_predict(coordinates_radians)
154     df_aggregated["cluster"] = labels
155
156     print("\nCluster label counts:")
157     print(df_aggregated["cluster"].value_counts().sort_index())
158
159     unique_labels = np.unique(labels)
```

The latitude and longitude columns are first selected from the aggregated dataset and converted into a NumPy array.

After that, coordinate information is printed out.

Next, the coordinates are converted from degrees to radians using np.radians(). This conversion is necessary because the Haversine distance metric used by OPTICS expects geographical coordinates in radians. The first five converted coordinate pairs are printed to confirm the transformation.

The min_samples value is set to 20. This means that OPTICS considers the density around each point using at least 20 nearby observations.

The min_cluster_size value is set to 50, meaning that a detected cluster must contain at least 50 postal-code observations.

The Earth's radius is defined as 6371.01 kilometres. It is later used to convert angular Haversine distances into kilometres for the reachability plot.

The OPTICS model is then created using the Haversine distance metric.

The xi parameter is set to 0.05. This controls how significant a change in reachability distance must be for OPTICS to identify a cluster boundary.

The fit_predict() method applies the OPTICS algorithm to the coordinates in radians and returns a cluster label for every postal-code prefix.

These labels are added to the aggregated DataFrame as a new column named cluster.

np.unique(labels) extracts all distinct labels produced by OPTICS.

```
160
161     number_of_clusters = len(unique_labels) - ( 1 if -1 in unique_labels else 0)
162     number_of_noise_points = np.sum(labels == -1)
163     noise_percentage = (number_of_noise_points / len(labels)) * 100
164
165     print("\nNumber of clusters: ", number_of_clusters)
166     print("Number of noise points: ", number_of_noise_points)
167     print("Noise percentage: ", noise_percentage)
168
169     optic_ordering = optic.ordering_
170     ordered_reachability = (optic.reachability_[optic_ordering])
171     reachability_km = (ordered_reachability * earth_radius)
172
173     finite_values = np.isfinite(reachability_km)
174     ordering_positions = np.arange(len(optic_ordering))
175
176     plt.scatter(optic_ordering[finite_values], reachability_km[finite_values], edgecolors='black',marker='o',
177                 c='royalblue',alpha=0.7)
178     plt.title("Reachability Plot")
179     plt.xlabel("Points in OPTICS Ordering")
180     plt.ylabel("Reachability Distance (km)")
181     plt.grid(True)
182     plt.savefig("reachability_plot.png")
183     plt.show()
184
```

Number of clusters and # of noise points are printed out, and lastly noise percentage is printed out.

The program then retrieves the ordering created by OPTICS using the ordering_ attribute. It also creates an ordered sequence of observations based on their density relationships.

The reachability distances are obtained from the reachability_ attribute and rearranged according to the OPTICS ordering.

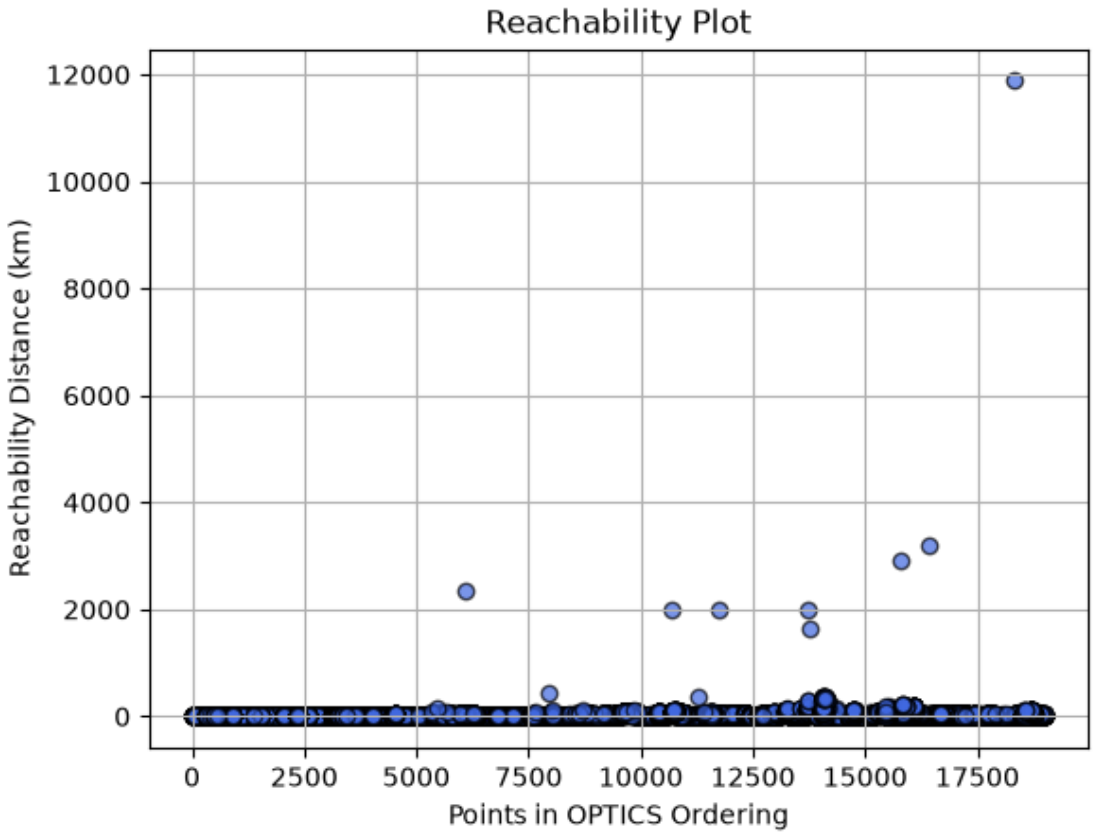
The np.isfinite() function creates a Boolean filter that keeps only finite reachability values.

The program also creates an array called ordering_positions.

The reachability plot is then created as a scatter plot.

The horizontal axis represents the observations according to their OPTICS ordering, while the vertical axis represents reachability distance in kilometres. Blue circular markers with black borders are used to make the observations easier to distinguish.

Here is the reachability plot graph:



The reachability plot provides a visual representation of the density structure discovered by OPTICS. Low reachability distances indicate dense areas where observations are located close to one another. These regions usually appear as valleys in the graph and may represent clusters. High reachability distances indicate sparse areas, transitions between clusters, or possible noise points. Peaks in the graph commonly represent boundaries between different clusters.

```
184
185  gdf = gpd.GeoDataFrame(df_aggregated, geometry=gpd.points_from_xy(df_aggregated["geolocation_lng"],
186                                                                    df_aggregated["geolocation_lat"]), crs='EPSG:4326')
187
188  gdf_map = gdf.to_crs(epsg=3857) # convert GeoDataFrame to real map background
189
190  fig, ax = plt.subplots(figsize=(12, 10))
```

Aggregated dataset is converted into a GeoDataFrame. It allows geographical objects such as points, lines, and polygons to be displayed and analyzed on a map.

```
191
192     gdf_map[gdf_map["cluster"] != -1].plot( # details for clusters
193         ax=ax,
194         column="cluster",
195         cmap="tab20",
196         edgecolor="none",
197         marker="o",
198         markersize=7,
199         alpha=0.8,
200         legend=False
201     )
202
203     gdf_map[gdf_map["cluster"] == -1].plot( # details for noises
204         ax=ax,
205         color='black',
206         markersize=4,
207         marker="x",
208         alpha=0.5,
209         label='noise'
210     )
211
212     brazil_area = gpd.GeoSeries(
213         data: [box(-74, -34, -34,  maxy: 6)],
214         crs="EPSG:4326"
215     ).to_crs(epsg=3857)
216
217     x_min, y_min, x_max, y_max = brazil_area.total_bounds
218
219     ax.set_xlim(x_min, x_max)
220     ax.set_ylim(y_min, y_max)
221
222     cx.add_basemap( # real map background
223         ax,
224         source=cx.providers.OpenStreetMap.Mapnik,
225         crs = gdf_map.crs,
226         zoom = 5
227     )
```

We created GeoDataFrame map for cluster points and outliers.

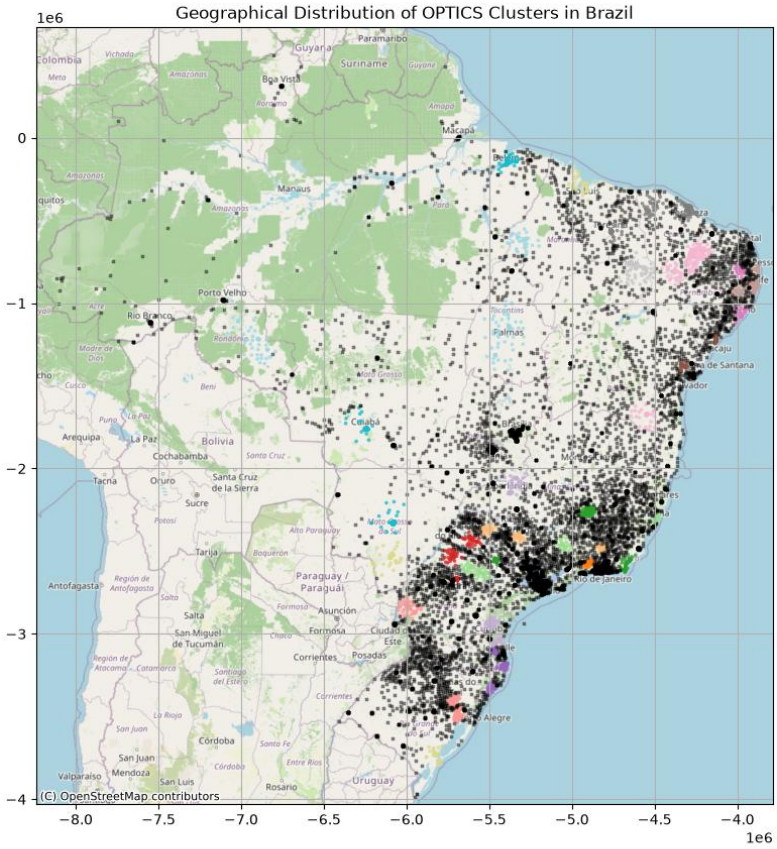
Box() function used to fit the points into area of Brazil. Limit added by using set_xlim and set_ylim to make sure keeping points inside Brazil area.

Add_basemap() used to add real map background.

After this we finally plotted geographical distribution of points which will be explained next page.

```
229     ax.set_title("Geographical Distribution of OPTICS Clusters in Brazil")
230     ax.grid(True)
231     plt.savefig("brazil.png")
232     plt.show()
```

Here is the output:



2. K-Modes Clustering

K-mode clustering is an unsupervised machine-learning algorithm used to group categorical data into k clusters (groups). The K-Modes clustering partitions the data into K mutually exclusive clusters. Unlike K-Means which uses distances between numbers K-Modes uses the number of mismatches between categorical values to decide how similar two data points are.

How it works?

Randomly select K amount of data points from the dataset to act as the starting clusters these are called "modes".

Check how similar each data point is to these clusters using the total number of mismatches and assign each data point to the cluster it matches the most.

Find the most common value for each cluster and update the cluster centers based on this.

Keep repeating these steps until cluster centers are stable.

Here is an python example that explains how K-Modes work:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from kmodes.kmodes import KModes

data = np.array([
    ['A', 'B', 'C'],
    ['B', 'C', 'A'],
    ['C', 'A', 'B'],
    ['A', 'C', 'B'],
    ['A', 'A', 'B']
])

k = 2 # of clusters
np.random.seed(0)
modes = data[np.random.choice(data.shape[0], k, replace=False)]

clusters = np.zeros(data.shape[0], dtype=int)

for _ in range(10):
    for i, point in enumerate(data):
        distances = [np.sum(point != mode) for mode in modes]
        clusters[i] = np.argmin(distances)

for j in range(k):
    if np.any(clusters == j):
        modes[j] == pd.DataFrame(data[clusters == j]).mode().iloc[0].values

print("Cluster assignments:", clusters)
print("Cluster modes:", modes)
# ------------------------------------------------------------------------------
```

This example demonstrates how the K-Modes clustering algorithm can be applied to a small categorical dataset.

Libraries such as pandas, numpy, matplotlib and kmodes are used to implement this program.

NumPy is used to create and process the categorical dataset, Pandas is used to calculate the most common values within clusters, Matplotlib is used to visualize the elbow curve, and the KModes class provides the library implementation of the K-Modes algorithm.

The number of clusters is initially set to two. The random seed is set to zero using np.random.seed(0), ensuring that the same initial modes are selected whenever the program is executed. This makes the results reproducible.

Two rows are randomly selected from the dataset as the initial cluster modes. The np.random.choice() function chooses the row indices, while replace=False ensures that the same observation cannot be selected more than once.

An integer array named clusters is created to store the cluster assignment of each observation.

In the for loop, the np.sum() function counts the number of mismatching features. Smaller distance means that an observation is more similar to the mode.

np.argmin() function returns the index of the mode with the smallest distance.

The mode() function calculates the most frequently occurring category in each column.

The first row of the result is selected using .iloc[0], and .values converts it into a NumPy array.

The condition np.any(clusters == j) checks whether at least one observation belongs to cluster j.

After the iterations are completed, the cluster assignments and final cluster modes are printed.

```
cost = []
K = range(1,5)

for k in list(K):
    k_mode = KModes(n_clusters=k, init='random', n_init=5, verbose=1)
    k_mode.fit_predict(data)
    cost.append(k_mode.cost_)

plt.plot( *args: K, cost, 'x-')
plt.xlabel('No. of clusters')
plt.ylabel('Cost')
plt.title('Elbow Curve')
plt.show()
```

An empty list named cost is created.

K defines the different numbers of clusters that will be tested when creating the elbow curve.

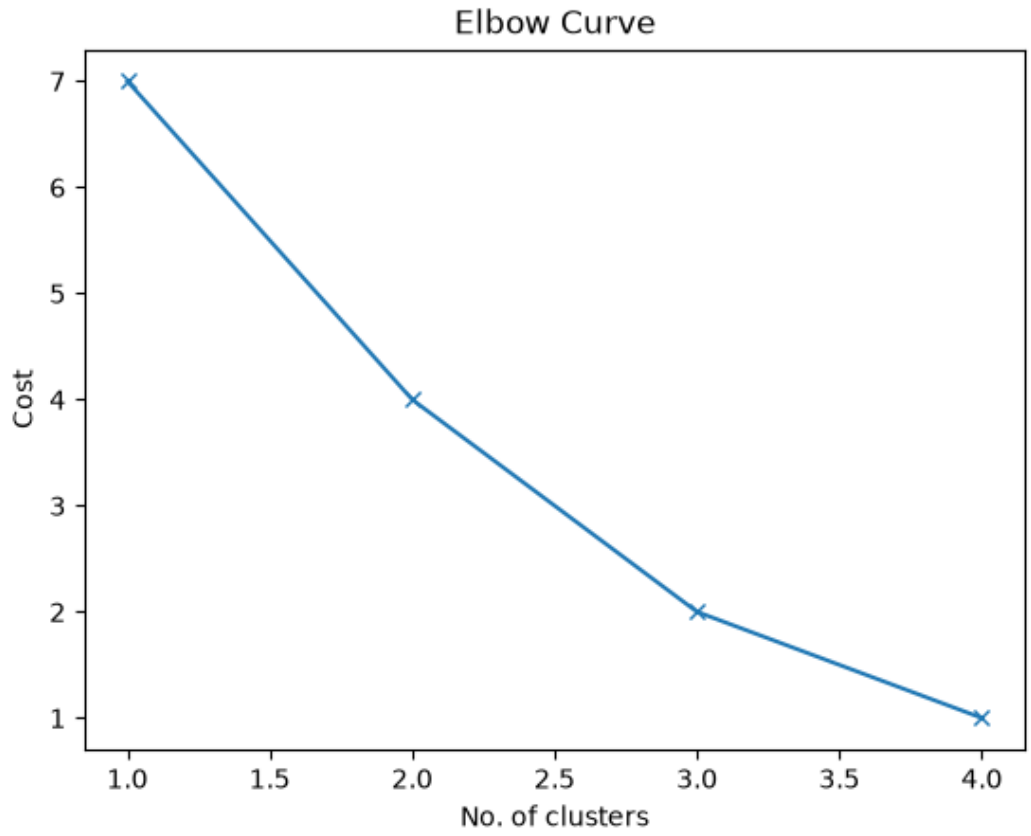
The n_clusters parameter determines how many clusters the algorithm should create.

The n_init=5 means that the algorithm must run the clustering process five times using different initial modes.

The fit_predict() method trains the model and assigns each observation to a cluster.

The cost is obtained through the cost_ attribute and added to the cost list.

Finally, we plotted the Elbow Curve which lets us see ideal amount of clusters to choose initially. Here is the graph:



The most suitable value of (k) is generally selected near the elbow point, where increasing the number of clusters stops producing a large reduction in cost.

As we can see from the graph there is an elbow-like shape at 2.0 and 3.0 Now it we can consider either 2.0 or 3.0 cluster.

2.1 K-Modes Clustering Program with Real Dataset

In this project, the K-Modes clustering algorithm was applied to the Car Evaluation dataset from UCI Machine Learning Repository.

The dataset contains categorical information about cars, including purchasing price, maintenance cost, number of doors, passenger capacity, luggage boot size, safety level, and an overall evaluation class.

The main objective of the project was to group cars according to their categorical characteristics without using the original evaluation class during model training. After clustering, the original class column was used to interpret the discovered clusters and examine whether the groups corresponded to the known car-evaluation categories.

Here is the python program:

```python
from kmodes.kmodes import KModes
import matplotlib.pyplot as plt
import pandas as pd

column_names = [
    "buying",
    "maint",
    "doors",
    "persons",
    "lug_boot",
    "safety",
    "class"
]

df = pd.read_csv( filepath_or_buffer: 'car.data', names=column_names)

print("First 5 rows of the car dataset:")
print(df.head())

print("\nLast 5 rows of the car dataset:")
print(df.tail())

print("\nDataset information:")
print(df.info())

print("\nDataset shape: ", df.shape)

print("\nDataset description:")
print(df.describe())

print("\nMissing values in the dataset:")
print(df.isnull().sum())
```

Kmodes library is added to implement K-Mode algorithm.

Pandas library used to read the dataset and matplot library is added for creating elbow curve.

Since the dataset doesnt define columns initially, we added column names manually.

After that, dataset information is printed out.

```
df_categorical = df[["buying", "maint", "doors", "persons", "lug_boot", "safety"]]
print("\nCategorical dataset\n")
print(df_categorical.head())
print("\n")

k = 4 # of clusters

modes = KModes(n_clusters=k, init='random', n_init=5, verbose=1, random_state=42)
cluster_labels = modes.fit_predict(df_categorical)

df["cluster"] = cluster_labels
print("\nDataset after adding cluster labels")
print(df.head(20))

print("\nHow many row each cluster has?")
print(df.value_counts(subset=['cluster']))

cluster_percentages = df["cluster"].value_counts(normalize=True).sort_index().mul(100).round(2)
cluster_counts = df["cluster"].value_counts().sort_index()

cluster_summary = pd.DataFrame({"row_count":cluster_counts, "cluster_percentage":cluster_percentages})
print("\nCluster summary:")
print(cluster_summary)

cluster_modes = pd.DataFrame(modes.cluster_centroids_, columns = df_categorical.columns)
cluster_modes.index.name ="cluster"

print("\nMode of each cluster:")
print(cluster_modes)

print("\nK-Modes cost:")
print(modes.cost_)

cluster_class_counts = pd.crosstab(df["cluster"], df["class"])
print("\nClass distribution inside each cluster:")
print(cluster_class_counts)
print("\n")
```

We only taked categorical columns from the dataset so K-Modes can be implemented.

Initially # of clusters are choosen as 4.

Kmodes() is implemented. The n_clusters=4 parameter instructs the algorithm to divide the dataset into four clusters.

The init="random" parameter means that the initial cluster modes are selected randomly.

Since random initialization can sometimes produce an unsuitable starting solution, n_init=5 causes the clustering process to run five times using different initial modes. The run with the lowest cost is retained as the final model.

After that, model is trained. The fit_predict() method performs two operations. First, it trains the K-Modes model using the selected categorical features. Second, it returns the cluster label assigned to every observation.

Cluster column is added to dataframe and it shows each rows cluster label.

By using value.counts() function, we found out how many row each cluster has.

Cluster percentages are found by multiplying each clusters # of rows and rounding up the number.

By using cluster percentages and cluster count, cluster_summary is created. Here is the output:

```
Cluster summary:
         row_count  cluster_percentage
cluster
0              591               34.20
1              502               29.05
2              275               15.91
3              360               20.83
```

After that, we find each clusters most frequent values in their columns. Modes is applied to each cluster centroids.

```
Mode of each cluster:
        buying  maint  doors persons lug_boot safety
cluster
0         high  vhigh      3       4      big    med
1          med   high  5more       4      med    low
2          med  vhigh  5more    more      big    med
3        vhigh    low      2       2      big   high
```

By using modes.cost_ , total cost of clustering operation is found out.

Class distribution of each cluster is found out using crosstab in the pandas library.

Output is at the next page:

```
Class distribution inside each cluster:
class    acc  good  unacc  vgood
cluster
0        165    24    390     12
1         78    11    388     25
2         92    24    147     12
3         49    10    285     16
```

```
class    acc  good  unacc  vgood  class_sum
cluster
0        165    24    390     12        591
1         78    11    388     25        502
2         92    24    147     12        275
3         49    10    285     16        360
```

```
72  cluster_class_counts["class_sum"] = cluster_class_counts.sum(axis=1)
73  print(cluster_class_counts)
74
75  cluster_class_percentages = pd.crosstab(df["cluster"], df["class"], normalize="index").mul(100).round(2)
76  print("\nEach class types percentage:")
77  print(cluster_class_percentages)
78
79  dominant_percentages = cluster_class_percentages.max(axis=1)
80  dominant_classes = cluster_class_percentages.idxmax(axis=1)
81  cluster_class_summary = pd.DataFrame({"dominant_class": dominant_classes, "dominant_percentage": dominant_percentages})
```

Each clusters class percentages are found after multiplying the # of classes in their columns. Here is the output:

```
Each class types percentage:
class      acc  good  unacc  vgood
cluster
0        27.92  4.06  65.99   2.03
1        15.54  2.19  77.29   4.98
2        33.45  8.73  53.45   4.36
3        13.61  2.78  79.17   4.44
```

After that, each clusters most dominant class and its percentage is printed out.

Output at the next page:

```
Cluster Class summary:
         dominant_class  dominant_percentage
cluster
0                 unacc                65.99
1                 unacc                77.29
2                 unacc                53.45
3                 unacc                79.17
```

```python

print("\nCluster Class summary:")
print(cluster_class_summary)
print("\nIt seems that unacc is the most frequent class in each cluster\n")

cost = []
k_values = range(2,11)

for k in k_values:
    model = KModes(n_clusters=k, init="random", n_init=5, verbose=1, random_state=42)
    model.fit_predict(df_categorical)
    cost.append(model.cost_)

plt.plot( *args: k_values, cost, marker="o")
plt.title("K-Modes Elbow Method")
plt.xlabel('No. of clusters')
plt.ylabel('Cost')
plt.title('Elbow Curve')
plt.savefig('elbow_curve.png')
plt.show()
# by looking at elbow method, we can say 4 is the ideal amount of clusters

```

The elbow method was used to find out the ideal amount of clusters for the program.

First, an empty list called cost was created to store the cost produced by each model. The variable k_values was defined as range(2, 11), meaning that the program tested models containing between 2 and 10 clusters.

An empty list named cost is created.

K defines the different numbers of clusters that will be tested when creating the elbow curve.

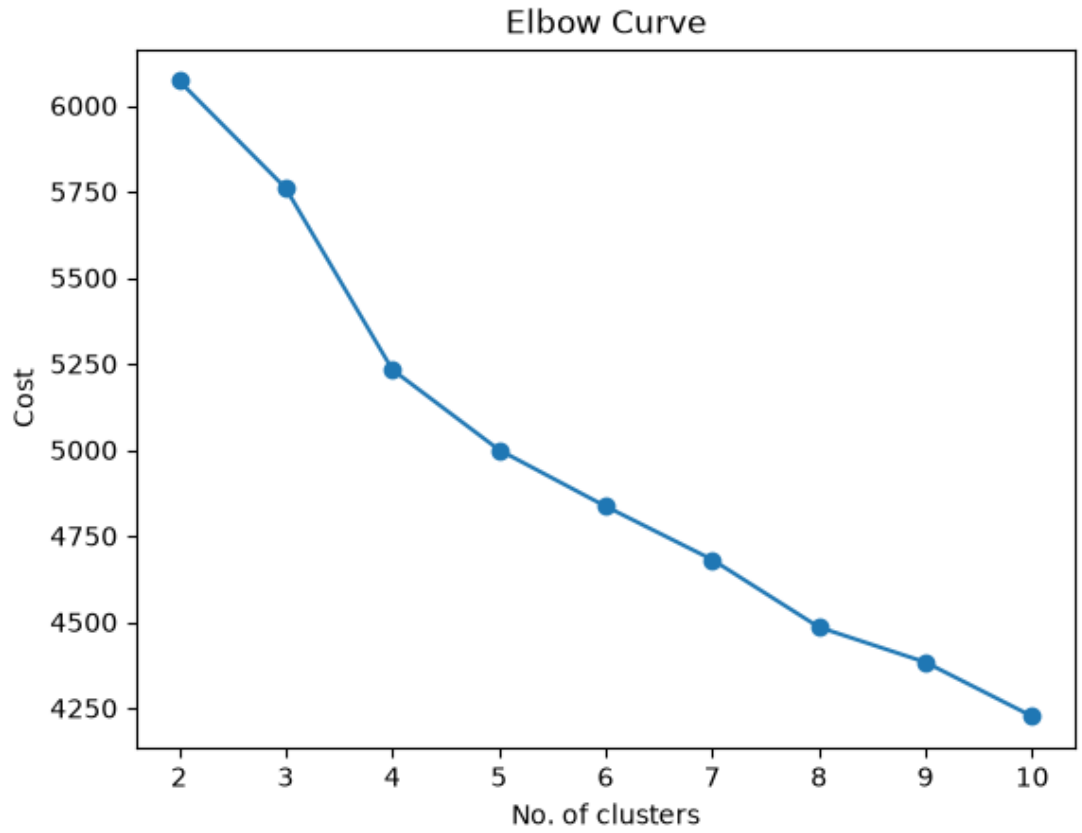
The n_clusters parameter determines how many clusters the algorithm should create.

The n_init=5 means that the algorithm must run the clustering process five times using different initial modes.

The fit_predict() method trains the model and assigns each observation to a cluster.

The cost is obtained through the cost_ attribute and added to the cost list.

Here is the elbow curve graph:

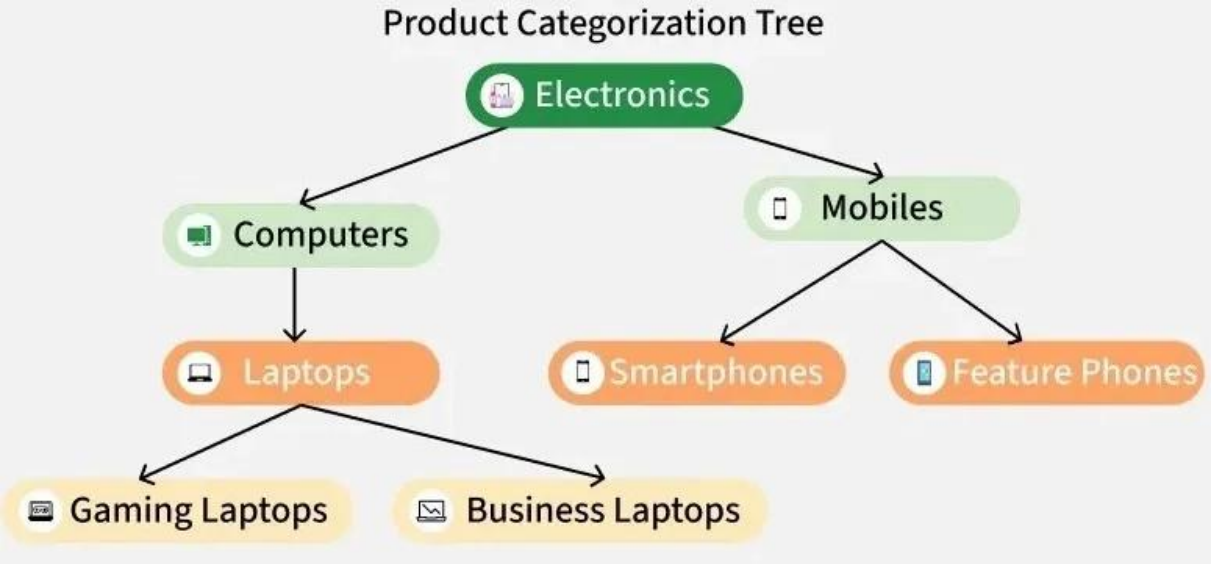


According to the resulting elbow curve, the most noticeable change in the rate of cost reduction occurred around four clusters. Therefore, (k=4) was selected as a reasonable number of clusters for the Car Evaluation dataset.

Finally, the elbow curve was saved as elbow_curve.png and displayed on the screen.

3.Divisive Clustering

Divisive Clustering is a type of hierarchical clustering that follows a top-down approach. It starts by placing all data points into one large cluster and then recursively splits that cluster into smaller ones based on differences or distances between the points. This process continues until each cluster contains only similar data points or meets a stopping condition. It is the opposite of agglomerative clustering, which builds clusters from the bottom up. It is is useful when we want to break down a broad category into smaller, meaningful groups.



How it works?

Begin with one big group ABCDEFGH.

Divide it into two groups ABC and DEFGH.

The group ABC is divided into A and BC while the group DEFGH is split into DEFG and H.

We continue dividing these new groups. BC is split into B and C, DEFG is divided into DE and so on.

Stop when all points are separated.

Now, we will implement Python example Divisive Clustering.

Code at next page:

py_ex.py

```python
import matplotlib.pyplot as plt

fruit_tree = {
    "All Fruits": {
        "Citrus": ["Orange", "Lemon"],
        "Non-Citrus": {
            "Berries": ["Strawberry", "Blueberry"],
            "Others": ["Apple", "Banana"]
        }
    }
}

print(fruit_tree)

def compute_positions(tree, x : int =0, y : int =0, dx : int =2):  2+ usages
    positions = {}
    if isinstance(tree, dict):
        all_child_positions = {}
        total_width = 0
        child_centers = []
        for key, subtree in tree.items():
            sub_pos, sub_width = compute_positions(subtree, x + total_width * dx, y - 2, dx)
            all_child_positions.update(sub_pos)
            child_center_x = sum(pos[0] for pos in sub_pos.values()) / len(sub_pos)
            child_centers.append((key, child_center_x))
            total_width += sub_width
        center_x = sum(center for _, center in child_centers) / len(child_centers)
        positions = {key: (center, y) for key, center in child_centers}
        positions.update(all_child_positions)
        return positions, total_width
    elif isinstance(tree, list):
        for i, item in enumerate(tree):
            positions[item] = (x + i * dx, y)
        return positions, len(tree)
    return {},0
```

```python
def extract_edges(tree, parent=None):  2+ usages
    edges = []
    if isinstance(tree, dict):
        for key, subtree in tree.items():
            if parent:
                edges.append((parent, key))
            edges.extend(extract_edges(subtree, key))
    elif isinstance(tree, list):
        for item in tree:
            if parent:
                edges.append((parent, item))
    return edges
```

```python
def plot_trees(tree):  1 usage
    positions, _ = compute_positions(tree)
    edges = extract_edges(tree)

    fig, ax = plt.subplots(figsize=(8,4))
    ax.axis('off')

    for parent, child in edges:
        if parent in positions and child in positions:
            x1, y1 = positions[parent]
            x2, y2 = positions[child]
            ax.plot( *args: [x1, x2], [y1, y2], 'k-')

    for node, (x,y) in positions.items():
        if node == "All Fruits":
            color = 'lightblue'
        elif node in ["Citrus", "Non-Citrus", "Berries", "Others"]:
            color = 'lightgreen'
        else:
            color = 'lightyellow'
        ax.text(x, y, node, ha='center', va='center', bbox = dict(boxstyle="round",

    plt.title( label: "Divisive Clustering Tree: Fruit Classification", fontsize=14)
    plt.tight_layout()
    plt.savefig("fruit_tree.png")
    plt.show()

plot_trees(fruit_tree)
```

In this example, a divisive hierarchical clustering structure is demonstrated using different types of fruit.

The program represents this process with a manually defined fruit hierarchy.

The hierarchy is then visualized as a tree using Matplotlib. The program calculates the position of each node, determines the connections between parent and child nodes, and displays the resulting structure using different colors.

Our first function is compute_positions(). This function calculates the x and y coordinates of every node in the fruit tree before Matplotlib draws it.

The function begins by creating an empty dictionary named positions.

When the current tree section is a dictionary, the function treats it as a cluster containing child branches. Three helper variables are created. The all_child_positions dictionary stores the coordinates of all nodes found below the current level. The total_width variable records how much horizontal space has already been used by previous branches.

The child_centers list stores the horizontal center of each child branch.

The function then processes every key and subtree in the dictionary.

For each subtree, the function calls itself recursively. The horizontal starting point is shifted according to the width occupied by earlier branches, while the vertical position is reduced by two units. This places child nodes below their parent nodes.

The recursive call returns two values: the positions calculated for the subtree and the width occupied by that subtree. The returned child positions are added to the complete collection of descendant positions.

The horizontal center of each child branch is then calculated by averaging the x-coordinates of all nodes inside that branch.

The cluster name and its calculated center are added to the child_centers list.

After all child branches have been processed, the function creates positions for the current cluster nodes.

Finally, the function returns the complete position dictionary and the total width occupied by the current branch.

When the current tree section is a list, the function treats the values as leaf nodes. Function uses enumerate() to obtain both the index and name of each fruit.

Each fruit is placed horizontally according to its index.

The function then returns the positions of the leaf nodes and the number of items in the list.

Overall, the compute_positions() function moves recursively through the hierarchy, places leaf nodes next to each other, positions cluster nodes above their descendants, and prevents branches from overlapping.

The extract_edges() function identifies all connections between parent nodes and child nodes in the hierarchical fruit structure.

The function receives two parameters. The tree parameter represents the current section of the hierarchy, while the optional parent parameter stores the name of the node located directly above the current section.

function starts by creating an empty list named edges.

The function then checks whether the current tree section is a dictionary.

For every key and subtree in the dictionary, the key represents the current node name, while the subtree represents the nodes below it.

If a parent node already exists, the function adds a connection between the parent and the current key to the edges list.

The function then calls itself recursively using the current subtree.

The edges returned by the recursive call are added to the current edge list using extend().

If the current tree section is a list, the function treats its values as leaf nodes.

The function loops through every item in the list. If a parent exists, it creates a connection between the parent cluster and the fruit.

After all dictionary branches and list items have been processed, the function returns the complete edges list.

The plot_trees() function is responsible for drawing the complete hierarchical fruit tree. It uses the node positions calculated by compute_positions() and the parent-child relationships extracted by extract_edges() to create a visual representation of the divisive clustering structure.

The variable positions stores the location of each cluster and fruit, while the underscore _ is used to ignore the second returned value, which represents the width of the tree branch.

The function then extracts the connections between nodes.

After that, matplotlib figure and axis are created.

ax.axis('off') removes tick marks, numbers, and axis borders because they are unnecessary in a tree diagram.

The function then loops through every edge in the edges list.

For each parent-child pair, it first checks whether both nodes exist in the positions dictionary.

This ensures that valid coordinates are available before drawing a connection.

The coordinates of the parent and child nodes are then retrieved. Then, these points are used for plotting.

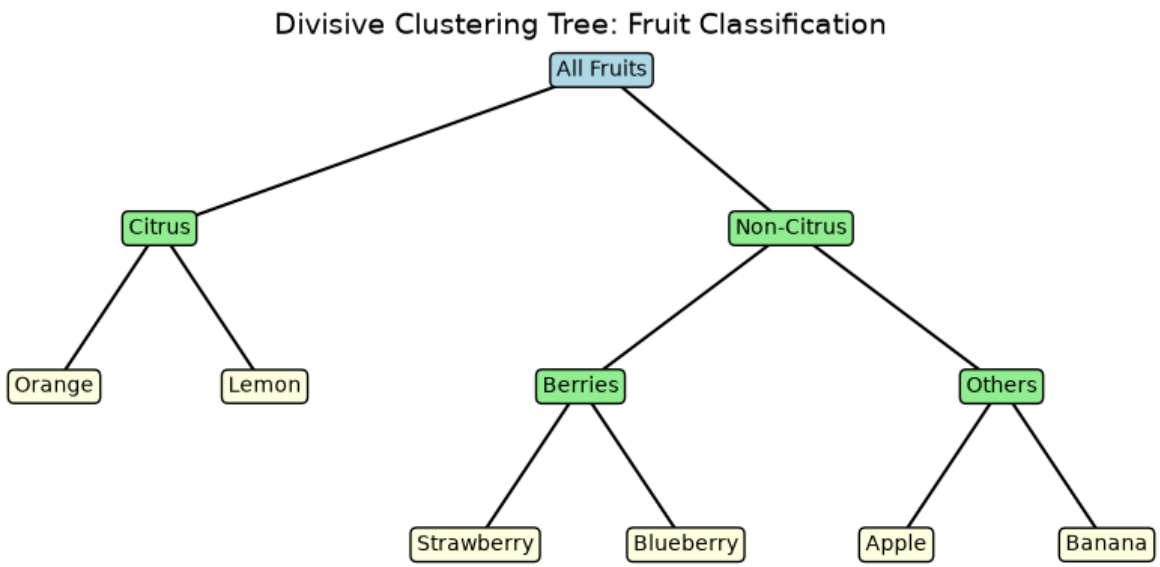
After drawing the edges, the function loops through every node and its position.

For each node, a color is chosen according to its role in the hierarchy.

Each node is displayed using ax.text().

The parameter ha='center' horizontally centers the text, while va='center' vertically centers it. The bbox argument draws a rounded box around the text, fills it with the selected background color, and adds a black border.

Lastly, we defined the title plotted the graph, output at the next page:



Overall, the plot_trees() function combines the calculated node positions and extracted parent-child relationships to produce a visual hierarchical diagram. It draws the branches, places the labels, applies different colors to different levels of the hierarchy, and saves the final result as an image.

4.Conclusion

In this report, three different clustering approaches were implemented and examined: OPTICS, K-Modes, and divisive hierarchical clustering. Although all three methods aim to group similar observations, they are designed for different types of data and represent clusters in different ways.

OPTICS was applied to geographical coordinate data from the Brazilian Olist Geolocation dataset. Before clustering, the dataset was cleaned, city and state names were standardized, and repeated postal-code prefixes were aggregated into representative geographical points. The latitude and longitude values were converted to radians, and the Haversine distance metric was used to measure realistic distances across the surface of the Earth.

K-Modes was applied to the Car Evaluation dataset, which consists entirely of categorical variables. Unlike numerical clustering algorithms such as K-Means, K-Modes compares observations by counting categorical mismatches and represents every cluster using the most frequent value of each feature.

The divisive hierarchical clustering example illustrated a different clustering structure. It began with all fruits inside one large cluster and divided them into increasingly specific subgroups. The hierarchy was represented using a nested dictionary and visualized as a tree.

In conclusion, the implemented examples provided practical experience with density-based, categorical, and hierarchical clustering methods. Together, they showed that different clustering algorithms can reveal different forms of structure within unlabeled data and that no single algorithm is suitable for every clustering problem.